Solution: Coin Change

Introduction

Given an infinite supply of ‘n’ coin denominations and a total money amount, we are asked to find the total number of distinct ways to make up that amount.

Example:

```
Denominations: {1,2,3}
Total amount: 5
Output: 5
Explanation: There are five ways to make the change for '5', here are
those ways:
  1. {1,1,1,1,1}
  2. {1,1,1,2}
  3. {1,2,2}
  4. {1,1,3}
  5. {2,3}
```

Problem Statement

Given a number array to represent different coin denominations and a total amount 'T', we need to find all the different ways to make a change for 'T' with the given coin denominations. We can assume an infinite supply of coins, therefore, each coin can be chosen multiple times.

This problem follows the `"Unbounded Knapsack"` pattern.

Basic Solution

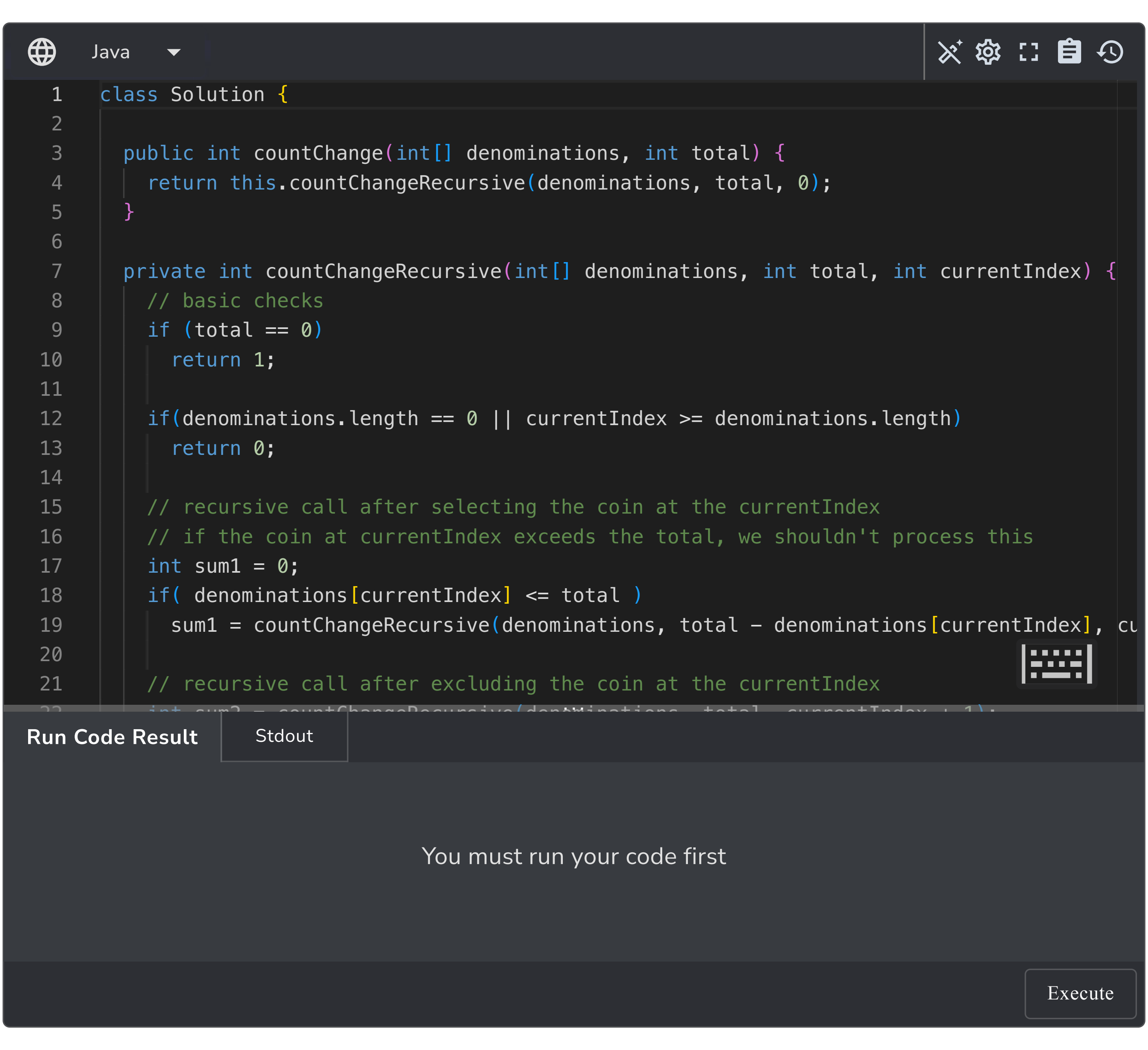
A basic brute-force solution could be to try all combinations of the given coins to select the ones that give a total sum of 'T'. This is what our algorithm will look like:

```
for each coin 'c'
  create a new set which includes one quantity of coin 'c' if it does not exceed 'T',
and
     recursively call to process all coins
  create a new set without coin 'c', and recursively call to process the remaining coins
return the count of sets who have a sum equal to 'T'
```

This problem is quite similar to `"Count of Subset Sum"`. The only difference here is that after including the item (i.e., coin), we recursively call to process all the items (including the current coin). In 'Count of Subset Sum', however, we were recursively calling to process only the remaining items.

Code

Here is the code for the brute-force solution:



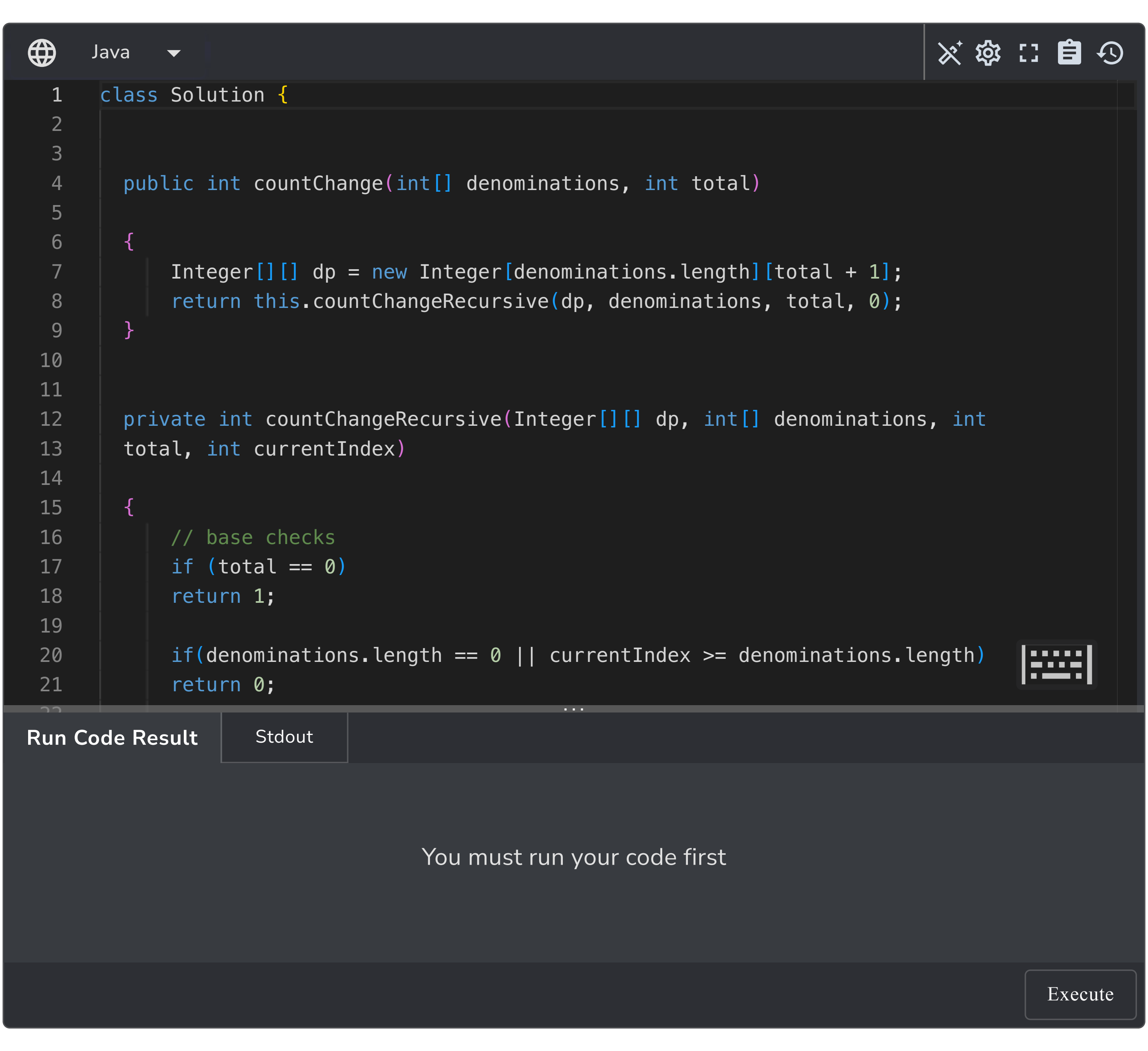
```java
class Solution {

  public int countChange(int[] denominations, int total) {
    return this.countChangeRecursive(denominations, total, 0);
  }

  private int countChangeRecursive(int[] denominations, int total, int currentIndex) {
    // basic checks
    if (total == 0)
      return 1;

    if(denominations.length == 0 || currentIndex >= denominations.length)
      return 0;

    // recursive call after selecting the coin at the currentIndex
    // if the coin at currentIndex exceeds the total, we shouldn't process this
    int sum1 = 0;
    if( denominations[currentIndex] <= total )
      sum1 = countChangeRecursive(denominations, total - denominations[currentIndex], cu

    // recursive call after excluding the coin at the currentIndex
```

The time complexity of the above algorithm is exponential $O(2^{C+T})$, where 'C' represents total coin denominations and 'T' is the total amount that we want to make change. The space complexity will be $O(C+T)$.

Let's try to find a better solution.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. We will be using a two-dimensional array to store the results of solved sub-problems. As mentioned above, we need to store results for every coin combination and for every possible sum:



```java
class Solution {


  public int countChange(int[] denominations, int total)

  {
      Integer[][] dp = new Integer[denominations.length][total + 1];
      return this.countChangeRecursive(dp, denominations, total, 0);
  }


  private int countChangeRecursive(Integer[][] dp, int[] denominations, int
  total, int currentIndex)

  {
      // base checks
      if (total == 0)
      return 1;

      if(denominations.length == 0 || currentIndex >= denominations.length)
      return 0;
```

Bottom-up Dynamic Programming

We will try to find if we can make all possible sums, with every combination of coins, to populate the array `dp[TotalDenominations][Total+1]`.

So for every possible total ‘t’ (0<= t <= Total) and for every possible coin index (0 <= index < denominations.length), we have two options:

1. Exclude the coin. Count all the coin combinations without the given coin up to the total 't' => `dp[index-1][t]`
2. Include the coin if its value is not more than ‘t’. In this case, we will count all the coin combinations to get the remaining total: `dp[index][t-denominations[index]]`

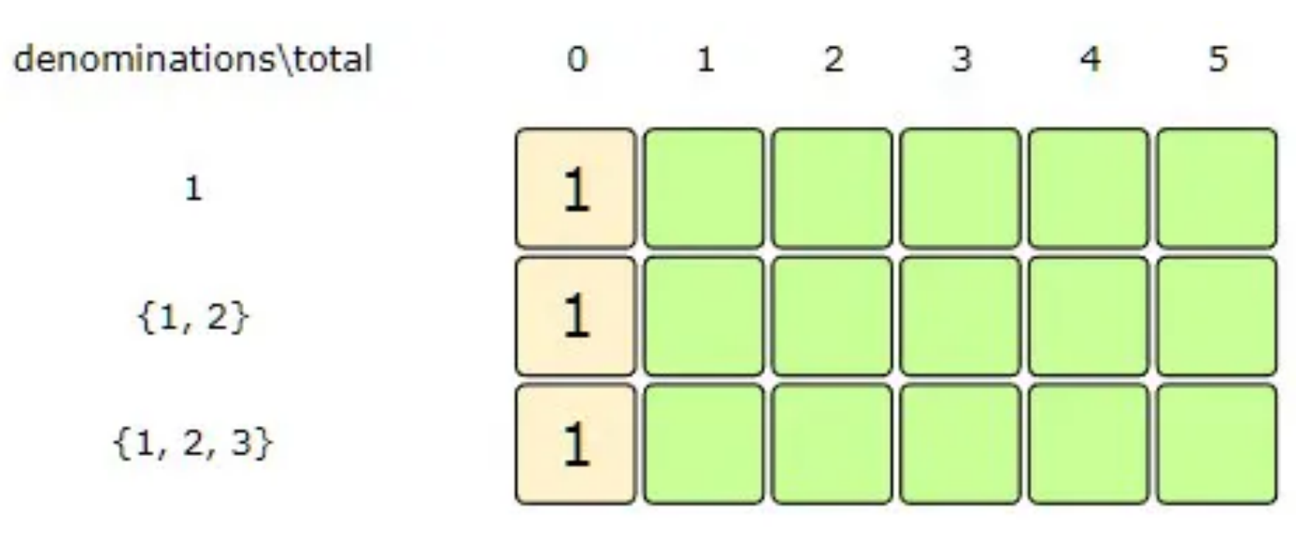
Finally, to find the total combinations, we will add both the above two values:

```
dp[index][t] = dp[index-1][t] + dp[index][t-denominations[index]]
```

Let’s draw this visually with the following example:

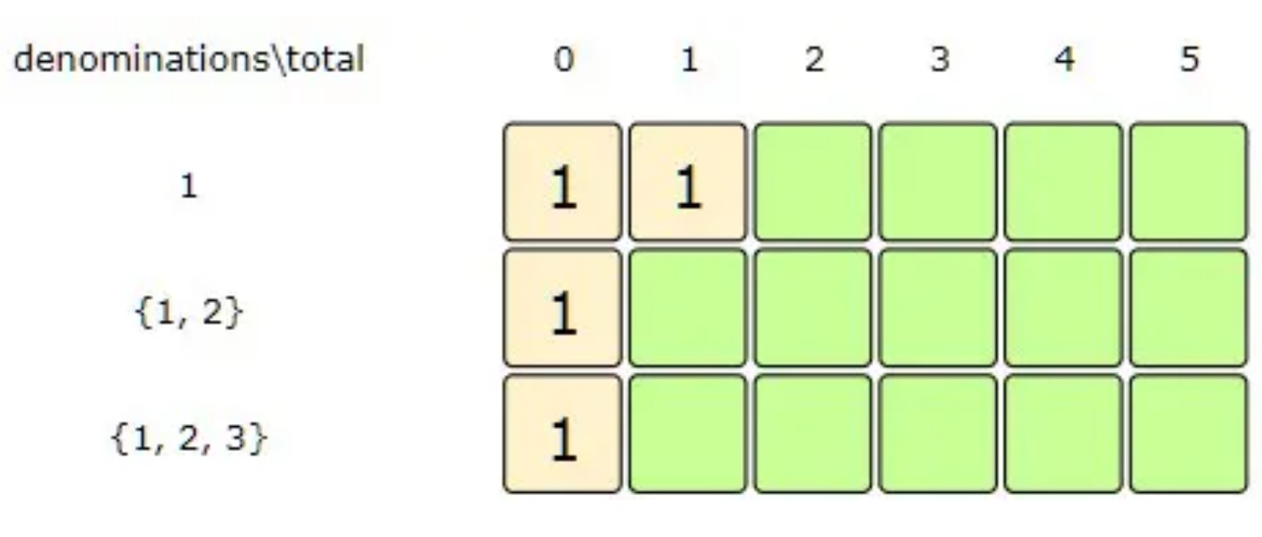
```
Denominations: [1, 2, 3]
Total: 5
```

Let’s start with our base case of zero total:



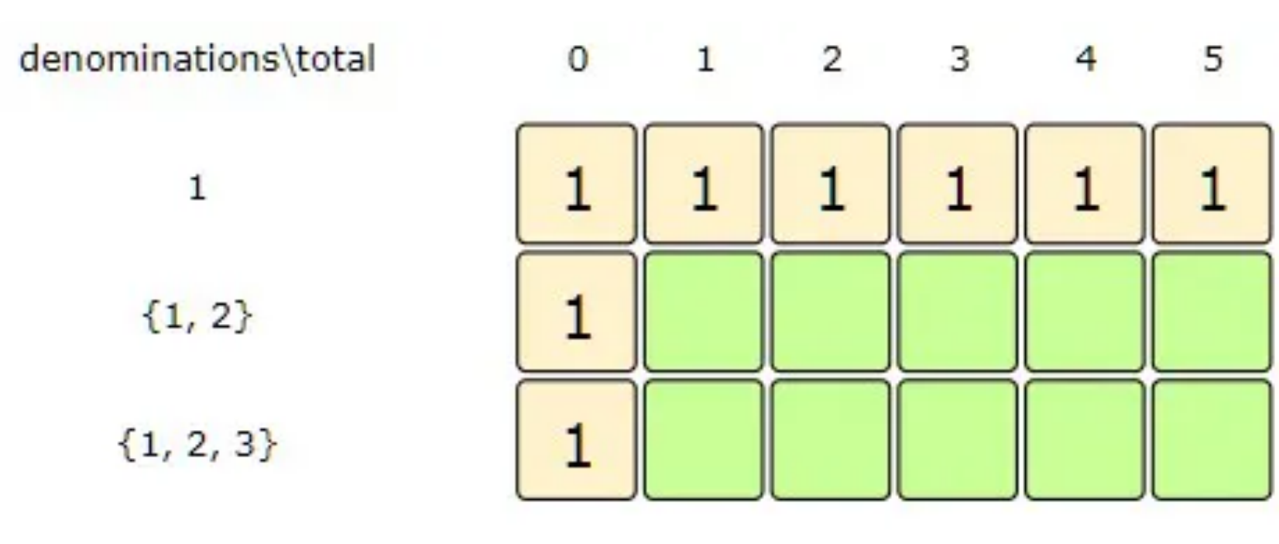
denominations\total	0	1	2	3	4	5
1	1					
{1, 2}	1					
{1, 2, 3}	1					

'0' total can always be found through an empty set



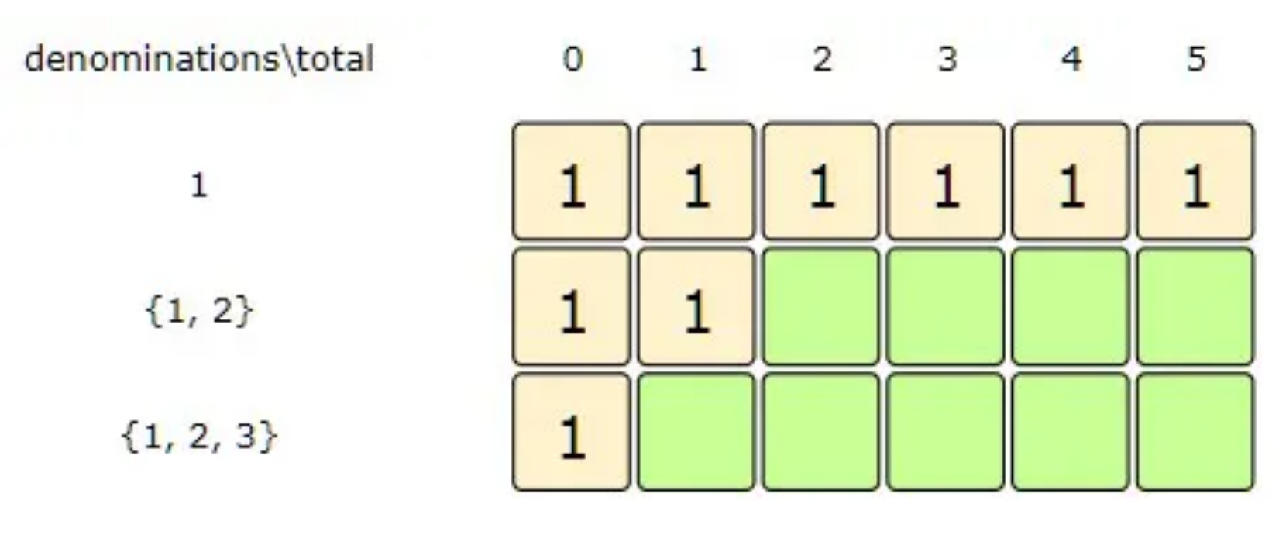
denominations\total	0	1	2	3	4	5
1	1	1				
{1, 2}	1					
{1, 2, 3}	1					

total: 1, index:0=> dp[index][total - denominations[index]], since index =0, we didn't consider dp[index-1][total]



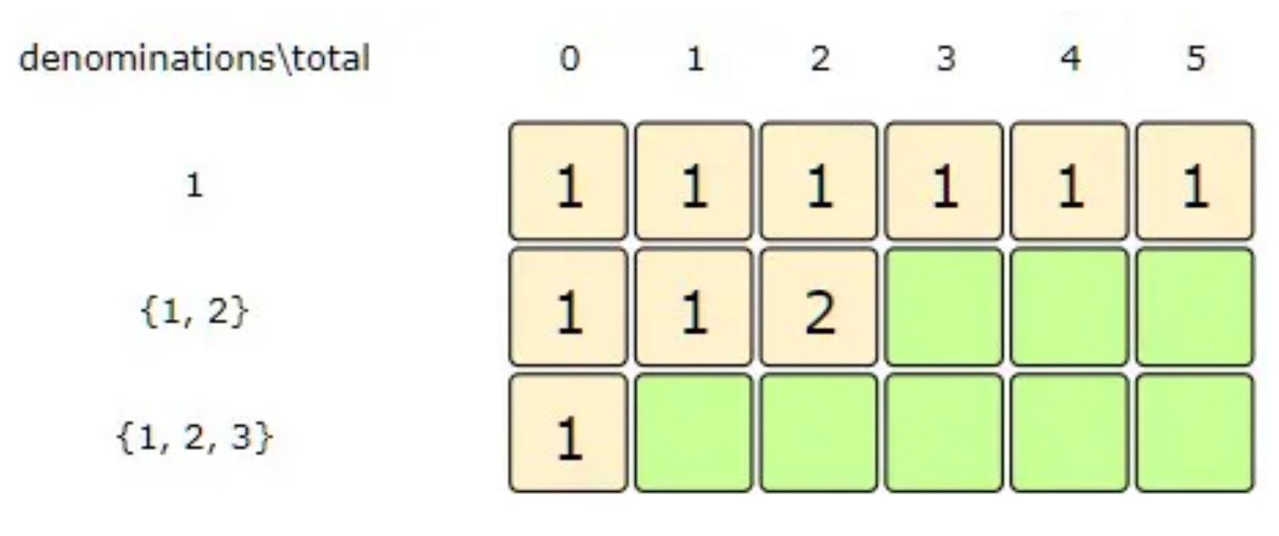
denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1					
{1, 2, 3}	1					

total: 2-5, index:0=> dp[index][total - denominations[index]], since index =0, we didn't consider dp[index-1][total]



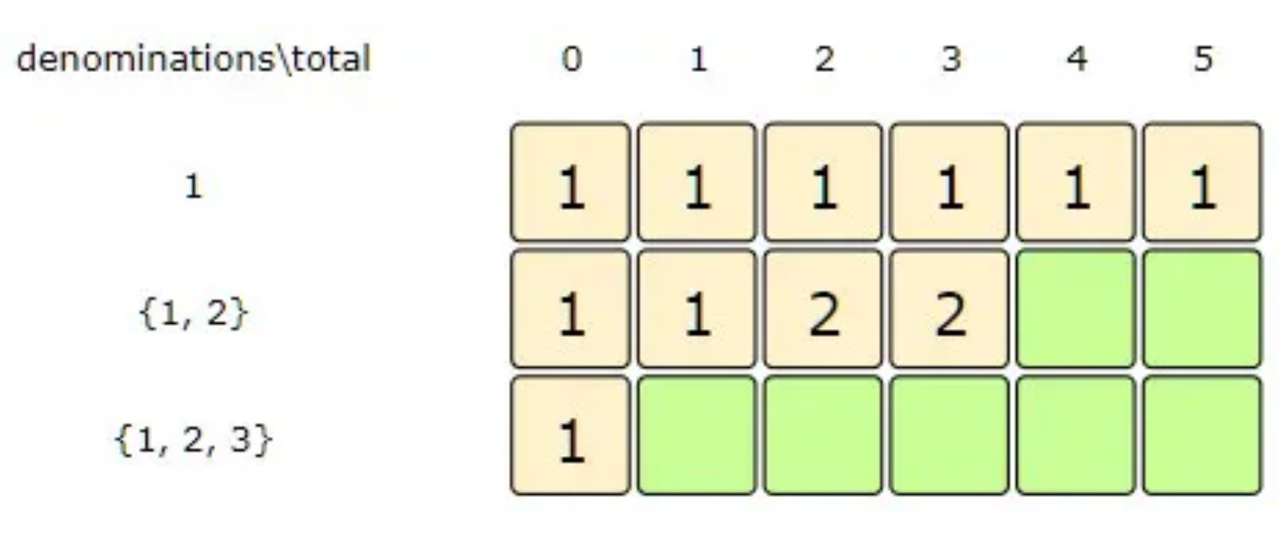
denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1				
{1, 2, 3}	1					

total: 1, index:1=> dp[index-1][total] , we didn't consider dp[index][total - denominations[index]], since 'total < denominations[index]'



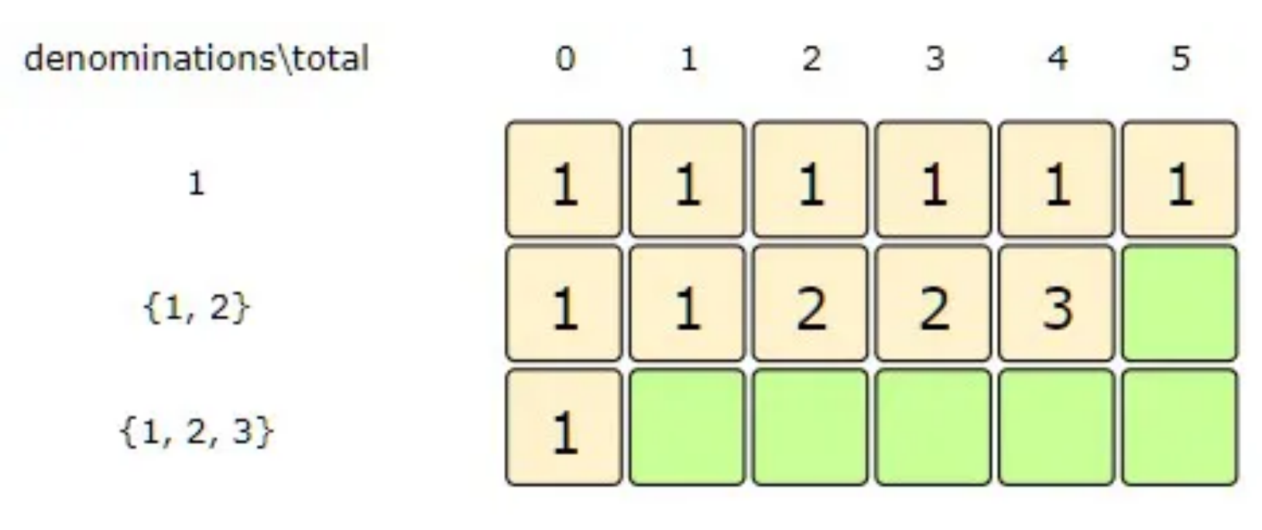
denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1	2			
{1, 2, 3}	1					

total: 2, index:1=> dp[index-1][total] + dp[index][total - denominations[index]]



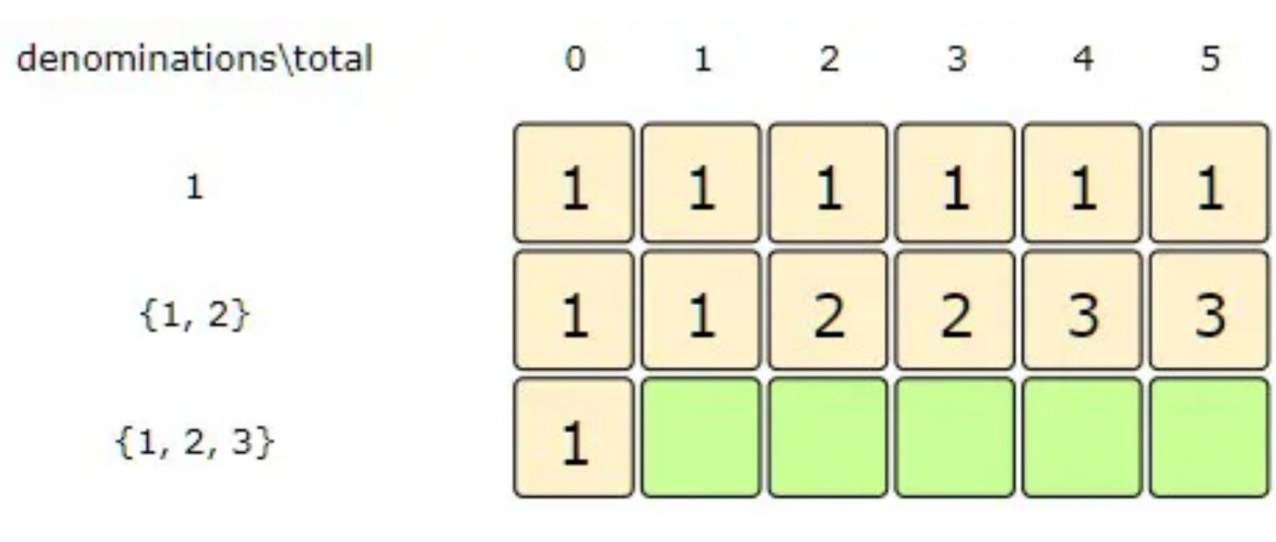
denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1	2	2		
{1, 2, 3}	1					

total: 3, index:1=> dp[index-1][total] + dp[index][total - denominations[index]]



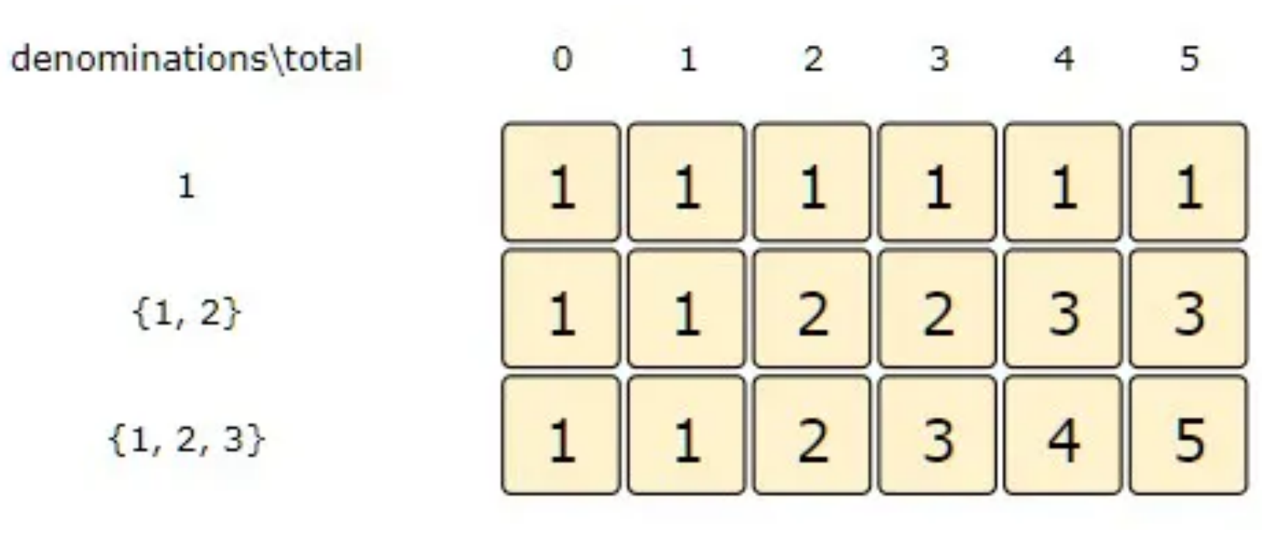
denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1	2	2	3	
{1, 2, 3}	1					

total: 4, index:1=> dp[index-1][total] + dp[index][total - denominations[index]]



denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1	2	2	3	3
{1, 2, 3}	1					

total: 5, index:1=> dp[index-1][total] + dp[index][total - denominations[index]]

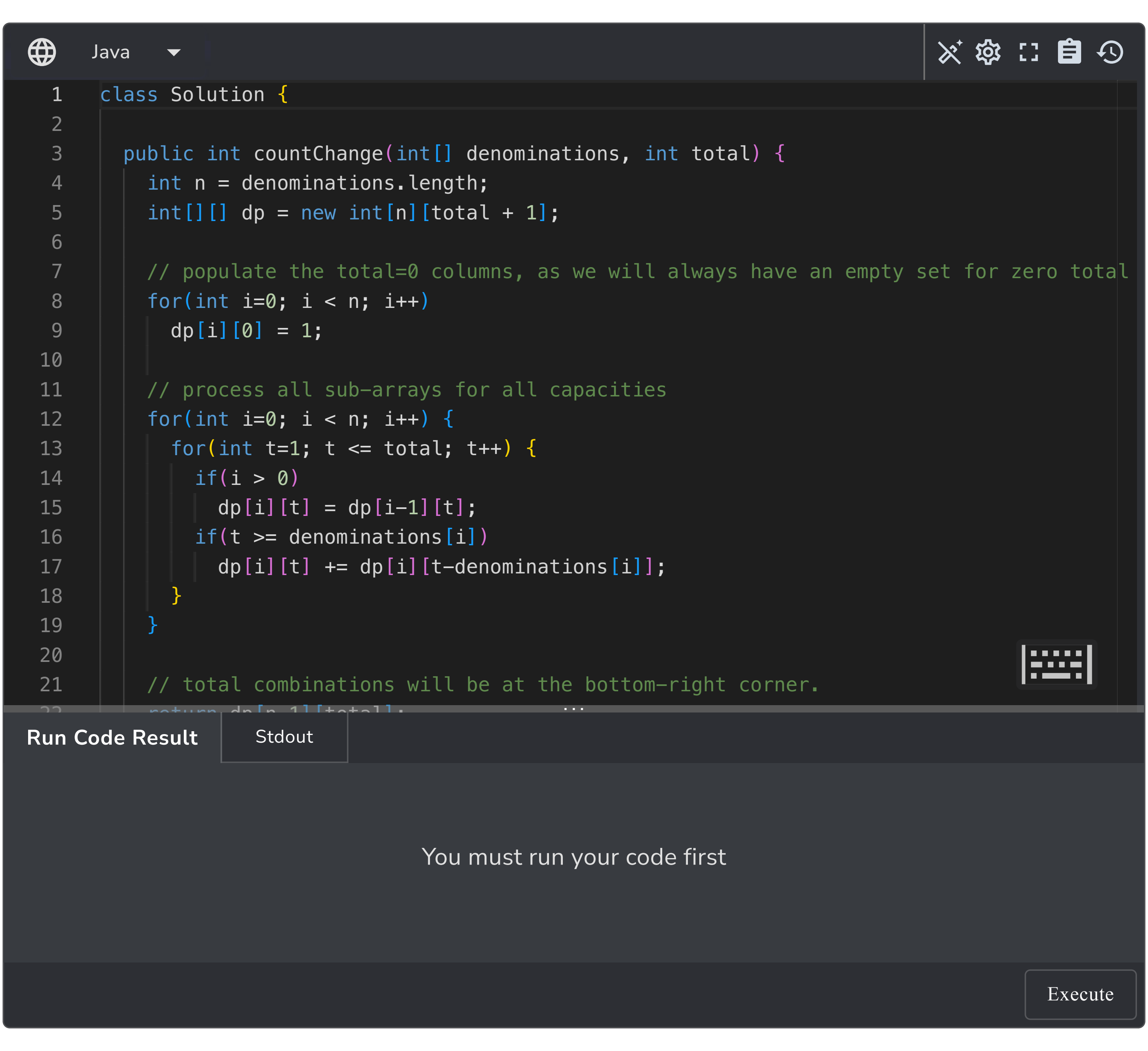


denominations\total	0	1	2	3	4	5
1	1	1	1	1	1	1
{1, 2}	1	1	2	2	3	3
{1, 2, 3}	1	1	2	3	4	5

total: 5, index:2=> dp[index-1][total] + dp[index][total - denominations[index]]

Code

Here is the code for our bottom-up dynamic programming approach:



```java
class Solution {

  public int countChange(int[] denominations, int total) {
    int n = denominations.length;
    int[][] dp = new int[n][total + 1];

    // populate the total=0 columns, as we will always have an empty set for zero total
    for(int i=0; i < n; i++)
      dp[i][0] = 1;

    // process all sub-arrays for all capacities
    for(int i=0; i < n; i++) {
      for(int t=1; t <= total; t++) {
        if(i > 0)
          dp[i][t] = dp[i-1][t];
        if(t >= denominations[i])
          dp[i][t] += dp[i][t-denominations[i]];
      }
    }

    // total combinations will be at the bottom-right corner.
```

The above solution has time and space complexity of $O(C*T)$, where 'C' represents total coin denominations and 'T' is the total amount that we want to make change.

← Previous
Coin Change
Next →
Minimum Coin Change
☐ Mark as Completed